

Lab 4: Data Imputation using an Autoencoder

In this lab, you will build and train an autoencoder to impute (or "fill in") missing data.

We will be using the Adult Data Set provided by the UCI Machine Learning Repository [1], available at <https://archive.ics.uci.edu/ml/datasets/adult>. The data set contains census record files of adults, including their age, marital status, the type of work they do, and other features.

Normally, people use this data set to build a supervised classification model to classify whether a person is a high income earner. We will not use the dataset for this original intended purpose.

Instead, we will perform the task of imputing (or "filling in") missing values in the dataset. For example, we may be missing one person's marital status, and another person's age, and a third person's level of education. Our model will predict the missing features based on the information that we do have about each person.

We will use a variation of a denoising autoencoder to solve this data imputation problem. Our autoencoder will be trained using inputs that have one categorical feature artificially removed, and the goal of the autoencoder is to correctly reconstruct all features, including the one removed from the input.

In the process, you are expected to learn to:

1. Clean and process continuous and categorical data for machine learning.
2. Implement an autoencoder that takes continuous and categorical (one-hot) inputs.
3. Tune the hyperparameters of an autoencoder.
4. Use baseline models to help interpret model performance.

[1] Dua, D. and Karra Taniskidou, E. (2017). UCI Machine Learning Repository [<http://archive.ics.uci.edu/ml>]. Irvine, CA: University of California, School of Information and Computer Science.

What to submit

Submit a PDF file containing all your code, outputs, and write-up. You can produce a PDF of your Google Colab file by going to File > Print and then save as PDF. The Colab

instructions have more information.

Do not submit any other files produced by your code.

Include a link to your colab file in your submission.

Colab Link

Include a link to your Colab file here. If you would like the TA to look at your Colab file in case your solutions are cut off, **please make sure that your Colab file is publicly accessible at the time of submission.**

Colab Link:

https://colab.research.google.com/github/sissi820li/APS360/blob/main/Lab%204/Lab%204%20-%20Data%20Imputation%20-%20Sissi820li.ipynb?authuser=2#scrollTo=IDQA_-dS1pq9

```
In [139... import csv
import numpy as np
import random
import torch
import torch.utils.data
import matplotlib.pyplot as plt
```

Part 0

We will be using a package called `pandas` for this assignment.

If you are using Colab, `pandas` should already be available. If you are using your own computer, installation instructions for `pandas` are available here:

<https://pandas.pydata.org/pandas-docs/stable/install.html>

```
In [140... import pandas as pd
```

Part 1. Data Cleaning [15 pt]

The adult.data file is available at <https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data>

The function `pd.read_csv` loads the adult.data file into a pandas dataframe. You can read about the pandas documentation for `pd.read_csv` at

https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html

```
In [141... header = ['age', 'work', 'fnlwgt', 'edu', 'yrelu', 'marriage', 'occupation',
'relationship', 'race', 'sex', 'capgain', 'caploss', 'workhr', 'country']
df = pd.read_csv(
    "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.csv",
    names=header,
    index_col=False)
```

/tmp/ipykernel_7346/1831985018.py:3: ParserWarning: Length of header or names does not match length of data. This leads to a loss of data with index_col=False.

```
df = pd.read_csv(
```

```
In [142... df.shape # there are 32561 rows (records) in the data frame, and 14 columns
```

```
Out[142... (32561, 14)
```

Part (a) Continuous Features [3 pt]

For each of the columns ["age", "yrelu", "capgain", "caploss", "workhr"], report the minimum, maximum, and average value across the dataset.

Then, normalize each of the features ["age", "yrelu", "capgain", "caploss", "workhr"] so that their values are always between 0 and 1. Make sure that you are actually modifying the dataframe `df`.

Like numpy arrays and torch tensors, pandas data frames can be sliced. For example, we can display the first 3 rows of the data frame (3 records) below.

```
In [143... df[:3] # show the first 3 records
```

```
Out[143...
```

	age	work	fnlwgt	edu	yrelu	marriage	occupation	relationship	race
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White

Alternatively, we can slice based on column names, for example `df["race"]`,

`df["hr"]` , or even index multiple columns like below.

```
In [144... subdf = df[["age", "yrelu", "capgain", "caploss", "workhr"]]
subdf[:3] # show the first 3 records
```

```
Out [144...   age  yrelu  capgain  caploss  workhr
0    39     13    2174         0        40
1    50     13         0         0        13
2    38         9         0         0        40
```

Numpy works nicely with pandas, like below:

```
In [145... np.sum(subdf["caploss"])
```

```
Out [145... np.int64(2842700)
```

Just like numpy arrays, you can modify entire columns of data rather than one scalar element at a time. For example, the code

```
df["age"] = df["age"] + 1
```

would increment everyone's age by 1.

```
In [146... print("AGE")
print("max: %d" % np.max(subdf["age"]))
print("min: %d" % np.min(subdf["age"]))
print("avg: %.2f" % np.average(subdf["age"]))
```

```
AGE
max: 90
min: 17
avg: 38.58
```

```
In [147... print("YREDU")
print("max: %d" % np.max(subdf["yrelu"]))
print("min: %d" % np.min(subdf["yrelu"]))
print("avg: %.2f" % np.average(subdf["yrelu"]))
```

```
YREDU
max: 16
min: 1
avg: 10.08
```

```
In [148... print("CAPGAIN")
print("max: %d" % np.max(subdf["capgain"]))
```

```
print("min: %d" % np.min(subdf["capgain"]))
print("avg: %.2f" % np.average(subdf["capgain"]))
```

CAPGAIN
 max: 99999
 min: 0
 avg: 1077.65

```
In [149... print("CAPLOSS")
print("max: %d" % np.max(subdf["caploss"]))
print("min: %d" % np.min(subdf["caploss"]))
print("avg: %.2f" % np.average(subdf["caploss"]))
```

CAPLOSS
 max: 4356
 min: 0
 avg: 87.30

```
In [150... print("WORKHR")
print("max: %d" % np.max(subdf["workhr"]))
print("min: %d" % np.min(subdf["workhr"]))
print("avg: %.2f" % np.average(subdf["workhr"]))
```

WORKHR
 max: 99
 min: 1
 avg: 40.44

```
In [151... features_to_normalize = ["age", "yrelu", "capgain", "caploss", "workhr"]
df[features_to_normalize] = (df[features_to_normalize] - df[features_to_normalize].min()) / (df[features_to_normalize].max() - df[features_to_normalize].min())
df.head(3)
```

```
Out [151... 
```

	age	work	fnlwgt	edu	yrelu	marriage	occupation	relationship
0	0.301370	State-gov	77516	Bachelors	0.800000	Never-married	Adm-clerical	Not-in-family
1	0.452055	Self-emp-not-inc	83311	Bachelors	0.800000	Married-civ-spouse	Exec-managerial	Husband
2	0.287671	Private	215646	HS-grad	0.533333	Divorced	Handlers-cleaners	Not-in-family

Part (b) Categorical Features [1 pt]

What percentage of people in our data set are male? Note that the data labels all have an unfortunate space in the beginning, e.g. " Male" instead of "Male".

What percentage of people in our data set are female?

```
In [152... # hint: you can do something like this in pandas
male = sum(df["sex"] == " Male")/32561
female = sum(df["sex"] == " Female")/32561

print(male*100, "% of people in the data set are male.")
print(female*100, "% of people in the data set are female.")
```

66.92054912318419 % of people in the data set are male.
33.07945087681583 % of people in the data set are female.

Part (c) [2 pt]

Before proceeding, we will modify our data frame in a couple more ways:

1. We will restrict ourselves to using a subset of the features (to simplify our autoencoder)
2. We will remove any records (rows) already containing missing values, and store them in a second dataframe. We will only use records without missing values to train our autoencoder.

Both of these steps are done for you, below.

How many records contained missing features? What percentage of records were removed?

```
In [153... contcols = ["age", "yrelu", "capgain", "caploss", "workhr"]
catcols = ["work", "marriage", "occupation", "edu", "relationship", "sex"]
features = contcols + catcols
df = df[features]
```

```
In [154... missing = pd.concat([df[c] == " ?" for c in catcols], axis=1).any(axis=1)
df_with_missing = df[missing]
df_not_missing = df[~missing]
```

```
In [155... missing_size = df_with_missing.shape[0]
original_total_size = df_with_missing.shape[0] + df_not_missing.shape[0]

print(missing_size, "records contained missing features.")
print((missing_size/original_total_size)*100, "% of records were removed.")
```

1843 records contained missing features.
5.660145572924664 % of records were removed.

Part (d) One-Hot Encoding [1 pt]

What are all the possible values of the feature "work" in `df_not_missing` ? You may find the Python function `set` useful.

```
In [156... set(df_not_missing)
```

```
Out[156... {'age',
            'capgain',
            'caploss',
            'edu',
            'marriage',
            'occupation',
            'relationship',
            'sex',
            'work',
            'workhr',
            'yrelu'}
```

We will be using a one-hot encoding to represent each of the categorical variables. Our autoencoder will be trained using these one-hot encodings.

We will use the pandas function `get_dummies` to produce one-hot encodings for all of the categorical variables in `df_not_missing`.

```
In [157... data = pd.get_dummies(df_not_missing, columns=catcols, dtype=int)
```

```
In [158... data[:3]
```

```
Out[158...
```

	age	yrelu	capgain	caploss	workhr	work_ Federal- gov	work_ Local- gov	work_ Private	work_ Self- emp- inc
0	0.301370	0.800000	0.02174	0.0	0.397959	0	0	0	0
1	0.452055	0.800000	0.00000	0.0	0.122449	0	0	0	0
2	0.287671	0.533333	0.00000	0.0	0.397959	0	0	1	0

3 rows × 57 columns

Part (e) One-Hot Encoding [2 pt]

The dataframe `data` contains the cleaned and normalized data that we will use to train our denoising autoencoder.

How many **columns** (features) are in the dataframe `data` ?

Briefly explain where that number come from.

```
In [159... print("Number of columns:", data.shape[1])
```

Number of columns: 57

The number comes from the sum of 5 continuous features and 52 possible values of categorical features.

Part (f) One-Hot Conversion [3 pt]

We will convert the pandas data frame `data` into numpy, so that it can be further converted into a PyTorch tensor. However, in doing so, we lose the column label information that a panda data frame automatically stores.

Complete the function `get_categorical_value` that will return the named value of a feature given a one-hot embedding. You may find the global variables `cat_index` and `cat_values` useful. (Display them and figure out what they are first.)

We will need this function in the next part of the lab to interpret our autoencoder outputs. So, the input to our function `get_categorical_values` might not actually be "one-hot" -- the input may instead contain real-valued predictions from our neural network.

```
In [160... datanp = data.values.astype(np.float32)
```

```
In [161... cat_index = {} # Mapping of feature -> start index of feature in a record
cat_values = {} # Mapping of feature -> list of categorical values the featu

# build up the cat_index and cat_values dictionary
for i, header in enumerate(data.keys()):
    if "_" in header: # categorical header
        feature, value = header.split()
        feature = feature[:-1] # remove the last char; it is always an under
        if feature not in cat_index:
            cat_index[feature] = i
            cat_values[feature] = [value]
        else:
            cat_values[feature].append(value)

def get_onehot(record, feature):
    """
    Return the portion of `record` that is the one-hot encoding
```


of `feature`. For example, since the feature "work" is stored in the indices [5:12] in each record, calling `get\_range(record, "work")` is equivalent to accessing `record[5:12]`.

Args:

- record: a numpy array representing one record, formatted the same way as a row in `data.np`
- feature: a string, should be an element of `catcols`

"""

```
start_index = cat_index[feature]
stop_index = cat_index[feature] + len(cat_values[feature])
return record[start_index:stop_index]
```

```
def get_categorical_value(onehot, feature):
```

"""

Return the categorical value name of a feature given a one-hot vector representing the feature.

Args:

- onehot: a numpy array one-hot representation of the feature
- feature: a string, should be an element of `catcols`

Examples:

```
>>> get_categorical_value(np.array([0., 0., 0., 0., 0., 1., 0.]), "work")
'State-gov'
>>> get_categorical_value(np.array([0.1, 0., 1.1, 0.2, 0., 1., 0.]), "work")
'Private'
"""
```

```
# <----- TODO: WRITE YOUR CODE HERE ----->
```

```
# You may find the variables `cat_index` and `cat_values`
```

```
# (created above) useful.
```

```
return cat_values[feature][int(np.argmax(onehot))]
```

In [162... *# more useful code, used during training, that depends on the function
you write above*

```
def get_feature(record, feature):
```

"""

Return the categorical feature value of a record

"""

```
onehot = get_onehot(record, feature)
return get_categorical_value(onehot, feature)
```

```
def get_features(record):
```

"""

Return a dictionary of all categorical feature values of a record

"""

```
return { f: get_feature(record, f) for f in catcols }
```

Part (g) Train/Test Split [3 pt]

Randomly split the data into approximately 70% training, 15% validation and 15% test.

Report the number of items in your training, validation, and test set.

```
In [163... # set the numpy seed for reproducibility
# https://docs.scipy.org/doc/numpy/reference/generated/numpy.random.seed.html
np.random.seed(50)

# todo
np.random.shuffle(data)

train_index = int(data.shape[0] * 0.7)
val_index = int(data.shape[0] * 0.85)

train_set = data[:train_index]
val_set = data[train_index:val_index]
test_set = data[val_index:]

print("The number of entries in the training set is", train_set.shape[0])
print("The number of entries in the validation set is", val_set.shape[0])
print("The number of entries in the test set is", test_set.shape[0])
```

The number of entries in the training set is 21502
 The number of entries in the validation set is 4608
 The number of entries in the test set is 4608

Part 2. Model Setup [5 pt]

Part (a) [4 pt]

Design a fully-connected autoencoder by modifying the `encoder` and `decoder` below.

The input to this autoencoder will be the features of the `data`, with one categorical feature recorded as "missing". The output of the autoencoder should be the reconstruction of the same features, but with the missing value filled in.

Note: Do not reduce the dimensionality of the input too much! The output of your embedding is expected to contain information about ~11 features.

```
In [164... from torch import nn

class AutoEncoder(nn.Module):
    def __init__(self):
```

```

super(AutoEncoder, self).__init__()
self.encoder = nn.Sequential(
    nn.Linear(57, 30), # TODO -- FILL OUT THE CODE HERE!
    nn.ReLU(),
    nn.Linear(30, 15)
)
self.decoder = nn.Sequential(
    nn.Linear(15, 30), # TODO -- FILL OUT THE CODE HERE!
    nn.ReLU(),
    nn.Linear(30, 57),
    nn.Sigmoid() # get to the range (0, 1)
)

def forward(self, x):
    x = self.encoder(x)
    x = self.decoder(x)
    return x

```

Part (b) [1 pt]

Explain why there is a sigmoid activation in the last step of the decoder.

(**Note:** the values inside the data frame `data` and the training code in Part 3 might be helpful.)

The sigmoid activation function is needed to scale the output from 0 to 1 since the the data frame values are all between 0 and 1.

Part 3. Training [18]

Part (a) [6 pt]

We will train our autoencoder in the following way:

- In each iteration, we will hide one of the categorical features using the `zero_out_random_features` function
- We will pass the data with one missing feature through the autoencoder, and obtain a reconstruction
- We will check how close the reconstruction is compared to the original data -- including the value of the missing feature

Complete the code to train the autoencoder, and plot the training and validation loss every few iterations. You may also want to plot training and validation "accuracy" every

few iterations, as we will define in part (b). You may also want to checkpoint your model every few iterations or epochs.

Use `nn.MSELoss()` as your loss function. (Side note: you might recognize that this loss function is not ideal for this problem, but we will use it anyway.)

```
In [165... def zero_out_feature(records, feature):
    """ Set the feature missing in records, by setting the appropriate
    columns of records to 0
    """
    start_index = cat_index[feature]
    stop_index = cat_index[feature] + len(cat_values[feature])
    records[:, start_index:stop_index] = 0
    return records

def zero_out_random_feature(records):
    """ Set one random feature missing in records, by setting the
    appropriate columns of records to 0
    """
    return zero_out_feature(records, random.choice(catcols))

def train(model, train_loader, valid_loader, num_epochs=5, learning_rate=1e-
    """ Training loop for Denoising Autoencoder. """
    torch.manual_seed(42)
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

    train_acc = np.zeros(num_epochs)
    train_loss = np.zeros(num_epochs)
    val_acc = np.zeros(num_epochs)
    val_loss = np.zeros(num_epochs)

    for epoch in range(num_epochs):
        total_loss = 0.0
        batch_count = 0
        #model.train() set to training mode

        for data in train_loader:
            optimizer.zero_grad()
            datam = zero_out_random_feature(data.clone()) #hide one category
            recon = model(datam)
            loss = criterion(recon, data) #check how close reconstruction is
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            batch_count += 1

        #model.eval() set to eval mode before calculating validation stats
```

```

train_acc[epoch] = get_accuracy(model, train_loader)
val_acc[epoch] = get_accuracy(model, valid_loader)
train_loss[epoch] = total_loss/(i + 1)
val_loss[epoch] = get_loss(model, valid_loader, criterion)
print(f"Epoch: {epoch + 1} | Train Loss: {train_loss[epoch]:.4f} | V

epochs = np.arange(1, num_epochs + 1)

#plot training curve
plt.title("Training vs Validation Accuracy")
plt.plot(epochs, train_acc, label="Train")
plt.plot(epochs, val_acc, label="Val")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(loc='best')
plt.show()

plt.title("Training Loss")
plt.plot(epochs, train_loss, label="Train")
plt.xlabel("Epochs")
plt.ylabel("Train Loss")
plt.legend(loc='best')
plt.show()

plt.title("Validation Loss")
plt.plot(epochs, val_loss, label="Val")
plt.xlabel("Epochs")
plt.ylabel("Validation Loss")
plt.legend(loc='best')
plt.show()

return train_acc, val_acc, train_loss, val_loss, epochs

```

Part (b) [3 pt]

While plotting training and validation loss is valuable, loss values are harder to compare than accuracy percentages. It would be nice to have a measure of "accuracy" in this problem.

Since we will only be imputing missing categorical values, we will define an accuracy measure. For each record and for each categorical feature, we determine whether the model can predict the categorical feature given all the other features of the record.

A function `get_accuracy` is written for you. It is up to you to figure out how to use the function. **You don't need to submit anything in this part.** To earn the marks, correctly plot the training and validation accuracy every few iterations as part of your training

curve.

```
In [166... def get_accuracy(model, data_loader):
    """Return the "accuracy" of the autoencoder model across a data set.
    That is, for each record and for each categorical feature,
    we determine whether the model can successfully predict the value
    of the categorical feature given all the other features of the
    record. The returned "accuracy" measure is the percentage of times
    that our model is successful.

    Args:
        - model: the autoencoder model, an instance of nn.Module
        - data_loader: an instance of torch.utils.data.DataLoader

    Example (to illustrate how get_accuracy is intended to be called.
    Depending on your variable naming this code might require
    modification.)

    >>> model = AutoEncoder()
    >>> vdl = torch.utils.data.DataLoader(data_valid, batch_size=256, sh
    >>> get_accuracy(model, vdl)
    """
    total = 0
    acc = 0
    for col in catcols:
        for item in data_loader: # minibatches
            inp = item.detach().numpy()
            out = model(zero_out_feature(item.clone(), col)).detach().numpy()
            for i in range(out.shape[0]): # record in minibatch
                acc += int(get_feature(out[i], col) == get_feature(inp[i], c
                total += 1
    return acc / total

def get_loss(model, data_loader, criterion):
    total_loss = 0
    i = 0
    for data in data_loader:
        datam = zero_out_random_feature(data.clone()) #zero out one categorical
        recon = model(datam)
        loss = criterion(recon, data)
        total_loss += loss.item()
        i += 1

    return total_loss/(i + 1)

def get_feature(record, feature):
    start = cat_index[feature]
    stop = cat_index[feature] + len(cat_values[feature])
    return np.argmax(record[start:stop])
```

Part (c) [4 pt]

Run your updated training code, using reasonable initial hyperparameters.

Include your training curve in your submission.

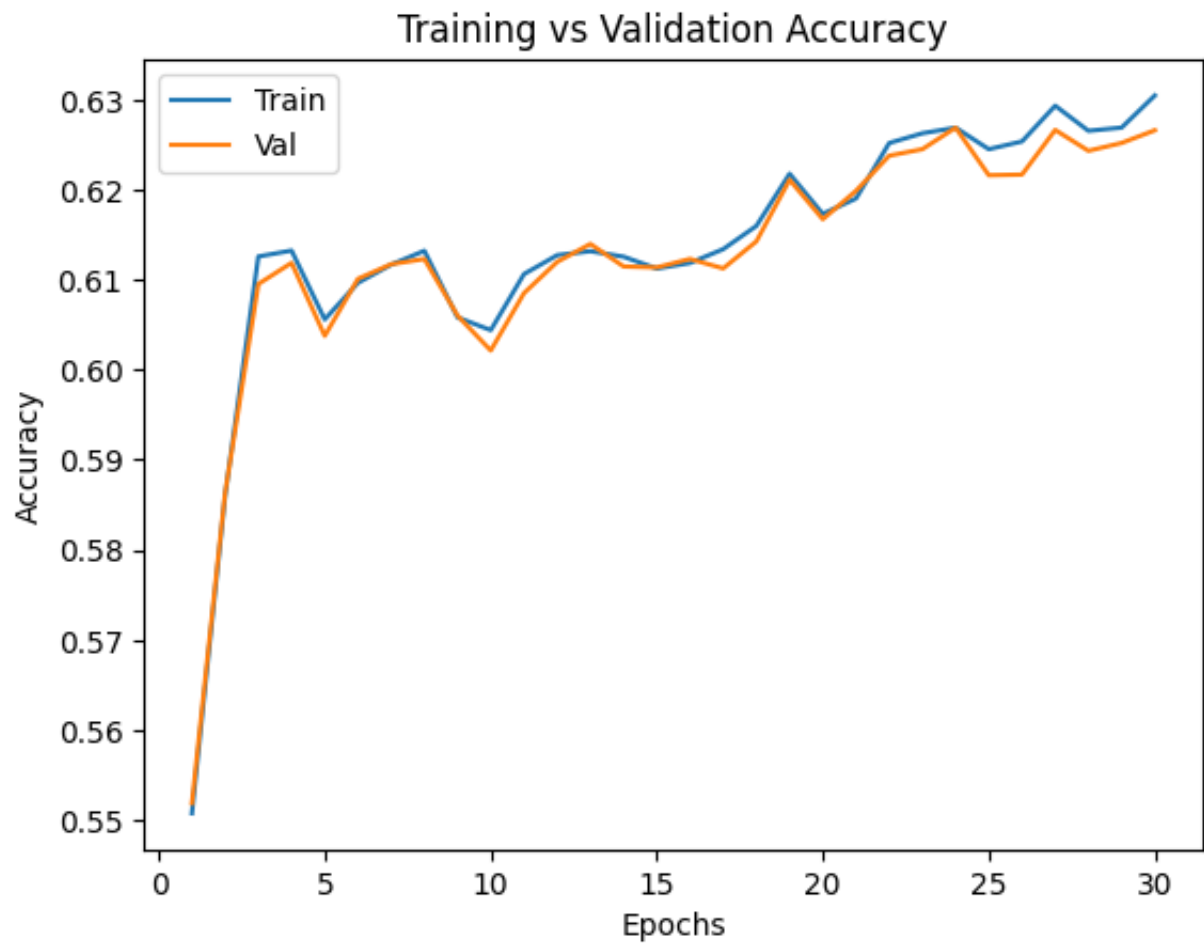
```
In [167... batch_size=64
train_loader = torch.utils.data.DataLoader(train_set, batch_size=batch_size,
valid_loader = torch.utils.data.DataLoader(val_set, batch_size=batch_size, r
test_loader = torch.utils.data.DataLoader(test_set, batch_size=batch_size, r

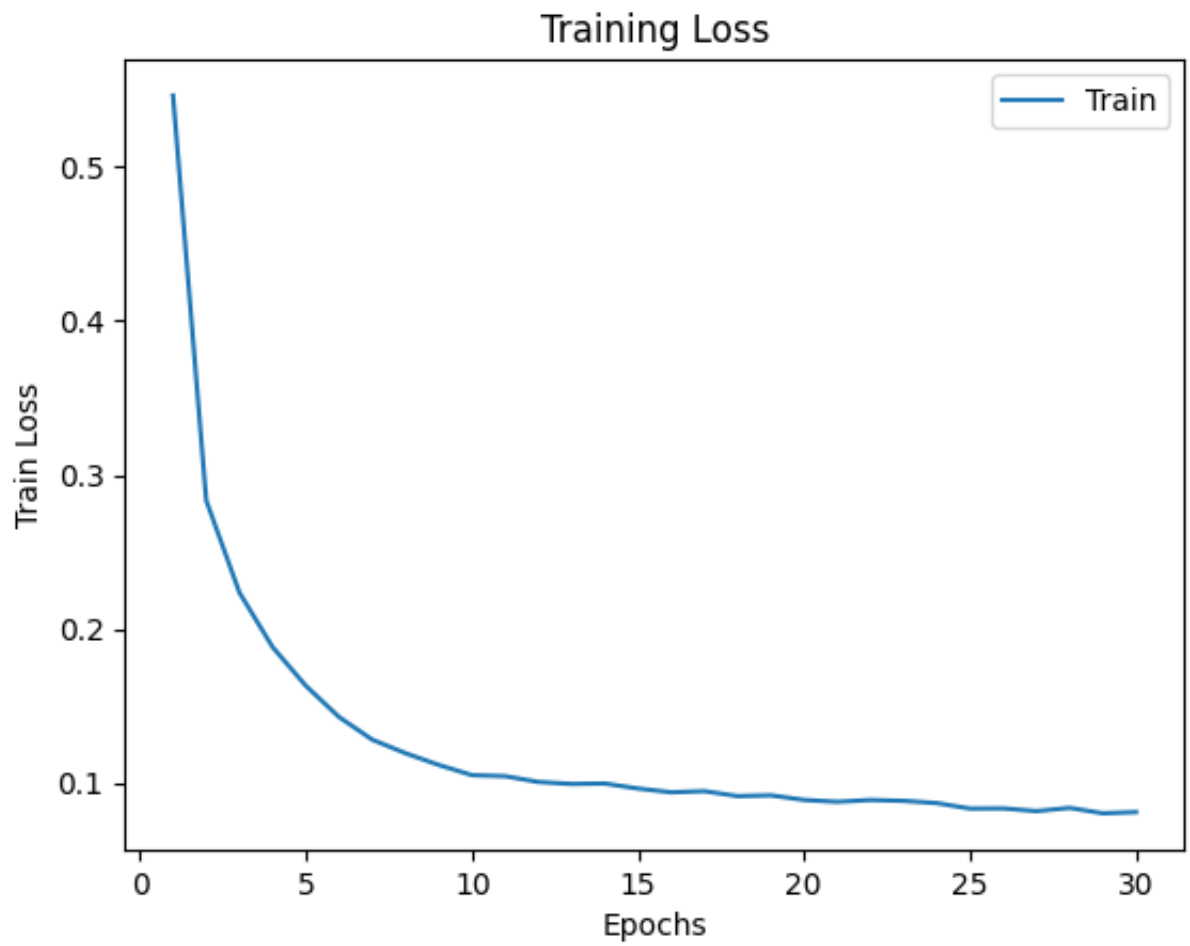
autoencoder1 = AutoEncoder()

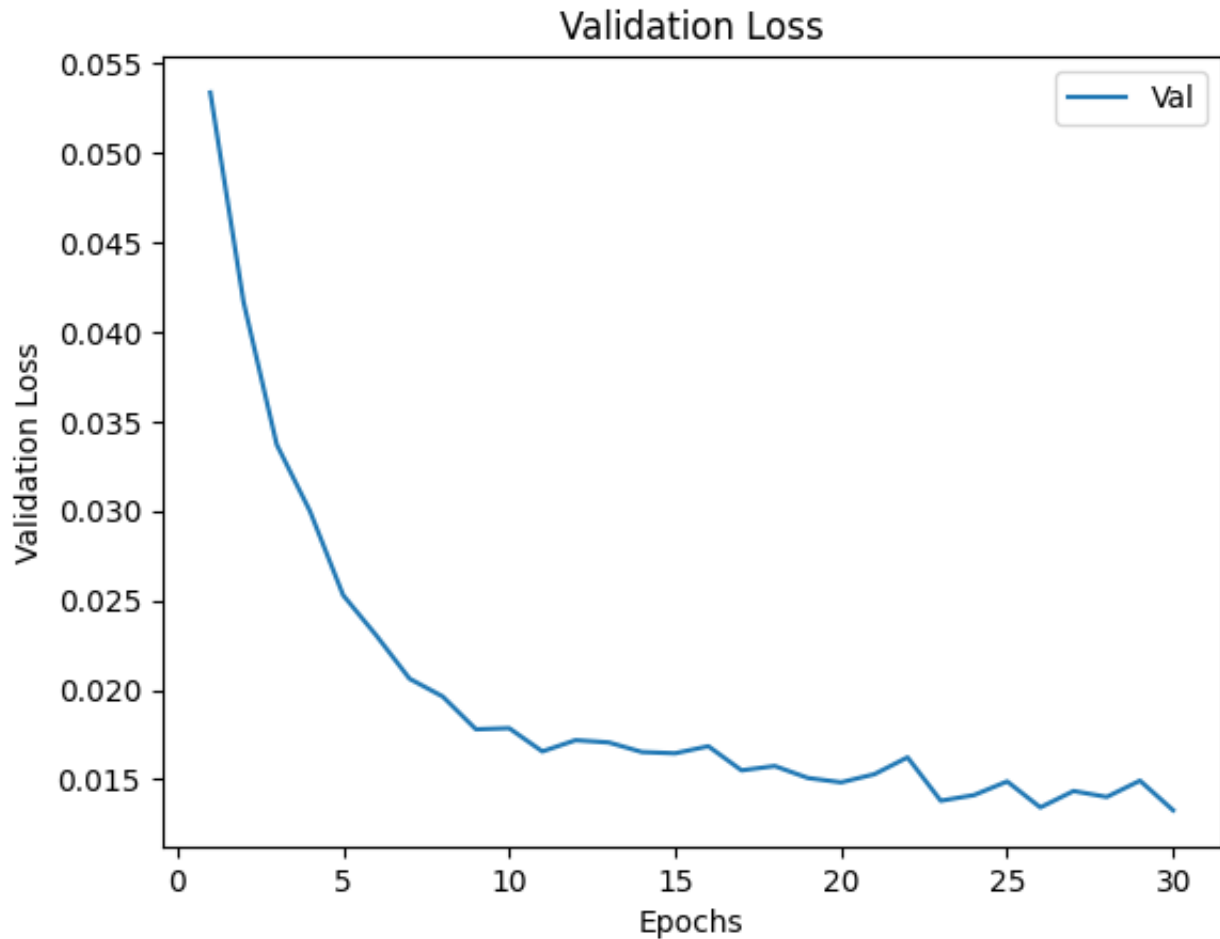
train_acc, val_acc, train_loss, val_loss, epochs = train(autoencoder1, train
```

```
Epoch: 1 | Train Loss: 0.5464 | Val Loss: 0.0534 | Train Acc: 0.5508 | Val A
cc: 0.5519
Epoch: 2 | Train Loss: 0.2834 | Val Loss: 0.0417 | Train Acc: 0.5860 | Val A
cc: 0.5863
Epoch: 3 | Train Loss: 0.2236 | Val Loss: 0.0337 | Train Acc: 0.6126 | Val A
cc: 0.6095
Epoch: 4 | Train Loss: 0.1881 | Val Loss: 0.0300 | Train Acc: 0.6132 | Val A
cc: 0.6119
Epoch: 5 | Train Loss: 0.1630 | Val Loss: 0.0253 | Train Acc: 0.6056 | Val A
cc: 0.6038
Epoch: 6 | Train Loss: 0.1428 | Val Loss: 0.0230 | Train Acc: 0.6097 | Val A
cc: 0.6101
Epoch: 7 | Train Loss: 0.1281 | Val Loss: 0.0206 | Train Acc: 0.6117 | Val A
cc: 0.6117
Epoch: 8 | Train Loss: 0.1193 | Val Loss: 0.0196 | Train Acc: 0.6132 | Val A
cc: 0.6123
Epoch: 9 | Train Loss: 0.1117 | Val Loss: 0.0178 | Train Acc: 0.6058 | Val A
cc: 0.6060
Epoch: 10 | Train Loss: 0.1050 | Val Loss: 0.0179 | Train Acc: 0.6044 | Val
Acc: 0.6021
Epoch: 11 | Train Loss: 0.1044 | Val Loss: 0.0166 | Train Acc: 0.6106 | Val
Acc: 0.6085
Epoch: 12 | Train Loss: 0.1006 | Val Loss: 0.0172 | Train Acc: 0.6127 | Val
Acc: 0.6120
Epoch: 13 | Train Loss: 0.0993 | Val Loss: 0.0171 | Train Acc: 0.6132 | Val
Acc: 0.6140
Epoch: 14 | Train Loss: 0.0996 | Val Loss: 0.0165 | Train Acc: 0.6126 | Val
Acc: 0.6115
Epoch: 15 | Train Loss: 0.0963 | Val Loss: 0.0165 | Train Acc: 0.6112 | Val
Acc: 0.6114
Epoch: 16 | Train Loss: 0.0939 | Val Loss: 0.0168 | Train Acc: 0.6119 | Val
Acc: 0.6123
Epoch: 17 | Train Loss: 0.0946 | Val Loss: 0.0155 | Train Acc: 0.6134 | Val
```

Acc: 0.6113
Epoch: 18 | Train Loss: 0.0914 | Val Loss: 0.0157 | Train Acc: 0.6160 | Val
Acc: 0.6143
Epoch: 19 | Train Loss: 0.0919 | Val Loss: 0.0151 | Train Acc: 0.6218 | Val
Acc: 0.6211
Epoch: 20 | Train Loss: 0.0889 | Val Loss: 0.0148 | Train Acc: 0.6173 | Val
Acc: 0.6167
Epoch: 21 | Train Loss: 0.0877 | Val Loss: 0.0153 | Train Acc: 0.6190 | Val
Acc: 0.6199
Epoch: 22 | Train Loss: 0.0889 | Val Loss: 0.0162 | Train Acc: 0.6252 | Val
Acc: 0.6238
Epoch: 23 | Train Loss: 0.0883 | Val Loss: 0.0138 | Train Acc: 0.6263 | Val
Acc: 0.6245
Epoch: 24 | Train Loss: 0.0869 | Val Loss: 0.0141 | Train Acc: 0.6269 | Val
Acc: 0.6269
Epoch: 25 | Train Loss: 0.0833 | Val Loss: 0.0149 | Train Acc: 0.6245 | Val
Acc: 0.6216
Epoch: 26 | Train Loss: 0.0834 | Val Loss: 0.0134 | Train Acc: 0.6254 | Val
Acc: 0.6217
Epoch: 27 | Train Loss: 0.0817 | Val Loss: 0.0143 | Train Acc: 0.6293 | Val
Acc: 0.6266
Epoch: 28 | Train Loss: 0.0838 | Val Loss: 0.0140 | Train Acc: 0.6265 | Val
Acc: 0.6243
Epoch: 29 | Train Loss: 0.0803 | Val Loss: 0.0149 | Train Acc: 0.6269 | Val
Acc: 0.6252
Epoch: 30 | Train Loss: 0.0810 | Val Loss: 0.0133 | Train Acc: 0.6305 | Val
Acc: 0.6266







Part (d) [5 pt]

Tune your hyperparameters, training at least 4 different models (4 sets of hyperparameters).

Do not include all your training curves. Instead, explain what hyperparameters you tried, what their effect was, and what your thought process was as you chose the next set of hyperparameters to try.

First model:

Batch size: 64 | Epochs: 30 | Learning rate: 0.001

Train acc: 0.6335 | Val Acc: 0.6275

Since the accuracy isn't good yet, first try a lower learning rate.

Second model: Batch size: 64 | Epochs: 30 | Learning rate: 0.0001

Train acc: 0.6132 | Val Acc: 0.6097

Since there is no improvement and the accuracy was worse, the takeaway is that a learning rate of 0.001 is better. For the next model, adjust the number of epochs instead.

Third model:

Batch size: 64 | Epochs: 50 | Learning rate: 0.001

Train acc: 0.6520 | Val Acc: 0.6461

More epochs resulted in better accuracy. Finally, try adjusting the batch size to see if it improves the results.

Fourth model:

Batch size: 128 | Epochs: 30 | Learning rate: 0.001

Train acc: 0.6301 | Val Acc: 0.6289

Adjusting the batch size didn't really change the overall accuracy of the model, so the batch size can stay at 64.

Best model:

Batch size: 64 | Epochs: 60 | Learning rate: 0.001

Train acc: 0.6680 | Val Acc: 0.6631

```
In [168... batch_size=64
train_loader = torch.utils.data.DataLoader(train_set, batch_size=batch_size,
valid_loader = torch.utils.data.DataLoader(val_set, batch_size=batch_size, r
test_loader = torch.utils.data.DataLoader(test_set, batch_size=batch_size, r

autoencoder1 = AutoEncoder()

train_acc, val_acc, train_loss, val_loss, epochs = train(autoencoder1, train
```

Epoch: 1 | Train Loss: 0.5386 | Val Loss: 0.0545 | Train Acc: 0.5325 | Val A
cc: 0.5332

Epoch: 2 | Train Loss: 0.2911 | Val Loss: 0.0421 | Train Acc: 0.5862 | Val A
cc: 0.5838

Epoch: 3 | Train Loss: 0.2161 | Val Loss: 0.0317 | Train Acc: 0.6116 | Val A
cc: 0.6097

Epoch: 4 | Train Loss: 0.1778 | Val Loss: 0.0281 | Train Acc: 0.6160 | Val A
cc: 0.6123

Epoch: 5 | Train Loss: 0.1618 | Val Loss: 0.0259 | Train Acc: 0.6080 | Val A

```
cc: 0.6034
Epoch: 6 | Train Loss: 0.1523 | Val Loss: 0.0245 | Train Acc: 0.6087 | Val A
cc: 0.6059
Epoch: 7 | Train Loss: 0.1422 | Val Loss: 0.0232 | Train Acc: 0.6072 | Val A
cc: 0.6054
Epoch: 8 | Train Loss: 0.1345 | Val Loss: 0.0221 | Train Acc: 0.6156 | Val A
cc: 0.6139
Epoch: 9 | Train Loss: 0.1295 | Val Loss: 0.0215 | Train Acc: 0.6150 | Val A
cc: 0.6135
Epoch: 10 | Train Loss: 0.1240 | Val Loss: 0.0207 | Train Acc: 0.6123 | Val
Acc: 0.6099
Epoch: 11 | Train Loss: 0.1178 | Val Loss: 0.0195 | Train Acc: 0.6175 | Val
Acc: 0.6143
Epoch: 12 | Train Loss: 0.1113 | Val Loss: 0.0186 | Train Acc: 0.6120 | Val
Acc: 0.6089
Epoch: 13 | Train Loss: 0.1067 | Val Loss: 0.0173 | Train Acc: 0.6177 | Val
Acc: 0.6155
Epoch: 14 | Train Loss: 0.1023 | Val Loss: 0.0171 | Train Acc: 0.6155 | Val
Acc: 0.6145
Epoch: 15 | Train Loss: 0.0987 | Val Loss: 0.0171 | Train Acc: 0.6174 | Val
Acc: 0.6165
Epoch: 16 | Train Loss: 0.0961 | Val Loss: 0.0166 | Train Acc: 0.6223 | Val
Acc: 0.6199
Epoch: 17 | Train Loss: 0.0935 | Val Loss: 0.0147 | Train Acc: 0.6251 | Val
Acc: 0.6229
Epoch: 18 | Train Loss: 0.0905 | Val Loss: 0.0150 | Train Acc: 0.6251 | Val
Acc: 0.6232
Epoch: 19 | Train Loss: 0.0890 | Val Loss: 0.0142 | Train Acc: 0.6243 | Val
Acc: 0.6233
Epoch: 20 | Train Loss: 0.0848 | Val Loss: 0.0142 | Train Acc: 0.6268 | Val
Acc: 0.6250
Epoch: 21 | Train Loss: 0.0848 | Val Loss: 0.0144 | Train Acc: 0.6262 | Val
Acc: 0.6251
Epoch: 22 | Train Loss: 0.0820 | Val Loss: 0.0136 | Train Acc: 0.6243 | Val
Acc: 0.6226
Epoch: 23 | Train Loss: 0.0815 | Val Loss: 0.0133 | Train Acc: 0.6318 | Val
Acc: 0.6293
Epoch: 24 | Train Loss: 0.0767 | Val Loss: 0.0134 | Train Acc: 0.6191 | Val
Acc: 0.6195
Epoch: 25 | Train Loss: 0.0749 | Val Loss: 0.0128 | Train Acc: 0.6292 | Val
Acc: 0.6264
Epoch: 26 | Train Loss: 0.0756 | Val Loss: 0.0123 | Train Acc: 0.6357 | Val
Acc: 0.6341
Epoch: 27 | Train Loss: 0.0724 | Val Loss: 0.0118 | Train Acc: 0.6258 | Val
Acc: 0.6254
Epoch: 28 | Train Loss: 0.0743 | Val Loss: 0.0120 | Train Acc: 0.6337 | Val
Acc: 0.6313
Epoch: 29 | Train Loss: 0.0735 | Val Loss: 0.0120 | Train Acc: 0.6347 | Val
Acc: 0.6337
Epoch: 30 | Train Loss: 0.0733 | Val Loss: 0.0133 | Train Acc: 0.6370 | Val
```

Acc: 0.6356
Epoch: 31 | Train Loss: 0.0701 | Val Loss: 0.0116 | Train Acc: 0.6363 | Val
Acc: 0.6329
Epoch: 32 | Train Loss: 0.0703 | Val Loss: 0.0120 | Train Acc: 0.6339 | Val
Acc: 0.6324
Epoch: 33 | Train Loss: 0.0698 | Val Loss: 0.0113 | Train Acc: 0.6333 | Val
Acc: 0.6301
Epoch: 34 | Train Loss: 0.0683 | Val Loss: 0.0117 | Train Acc: 0.6357 | Val
Acc: 0.6326
Epoch: 35 | Train Loss: 0.0685 | Val Loss: 0.0111 | Train Acc: 0.6360 | Val
Acc: 0.6341
Epoch: 36 | Train Loss: 0.0683 | Val Loss: 0.0128 | Train Acc: 0.6361 | Val
Acc: 0.6352
Epoch: 37 | Train Loss: 0.0680 | Val Loss: 0.0114 | Train Acc: 0.6378 | Val
Acc: 0.6357
Epoch: 38 | Train Loss: 0.0685 | Val Loss: 0.0114 | Train Acc: 0.6366 | Val
Acc: 0.6328
Epoch: 39 | Train Loss: 0.0677 | Val Loss: 0.0126 | Train Acc: 0.6418 | Val
Acc: 0.6391
Epoch: 40 | Train Loss: 0.0685 | Val Loss: 0.0115 | Train Acc: 0.6345 | Val
Acc: 0.6330
Epoch: 41 | Train Loss: 0.0695 | Val Loss: 0.0103 | Train Acc: 0.6405 | Val
Acc: 0.6391
Epoch: 42 | Train Loss: 0.0648 | Val Loss: 0.0106 | Train Acc: 0.6361 | Val
Acc: 0.6332
Epoch: 43 | Train Loss: 0.0663 | Val Loss: 0.0112 | Train Acc: 0.6413 | Val
Acc: 0.6391
Epoch: 44 | Train Loss: 0.0665 | Val Loss: 0.0099 | Train Acc: 0.6393 | Val
Acc: 0.6364
Epoch: 45 | Train Loss: 0.0657 | Val Loss: 0.0114 | Train Acc: 0.6423 | Val
Acc: 0.6407
Epoch: 46 | Train Loss: 0.0654 | Val Loss: 0.0113 | Train Acc: 0.6412 | Val
Acc: 0.6388
Epoch: 47 | Train Loss: 0.0634 | Val Loss: 0.0114 | Train Acc: 0.6381 | Val
Acc: 0.6361
Epoch: 48 | Train Loss: 0.0680 | Val Loss: 0.0113 | Train Acc: 0.6419 | Val
Acc: 0.6384
Epoch: 49 | Train Loss: 0.0659 | Val Loss: 0.0106 | Train Acc: 0.6427 | Val
Acc: 0.6384
Epoch: 50 | Train Loss: 0.0633 | Val Loss: 0.0114 | Train Acc: 0.6398 | Val
Acc: 0.6385
Epoch: 51 | Train Loss: 0.0668 | Val Loss: 0.0123 | Train Acc: 0.6415 | Val
Acc: 0.6393
Epoch: 52 | Train Loss: 0.0634 | Val Loss: 0.0113 | Train Acc: 0.6361 | Val
Acc: 0.6359
Epoch: 53 | Train Loss: 0.0669 | Val Loss: 0.0104 | Train Acc: 0.6404 | Val
Acc: 0.6395
Epoch: 54 | Train Loss: 0.0619 | Val Loss: 0.0110 | Train Acc: 0.6392 | Val
Acc: 0.6362
Epoch: 55 | Train Loss: 0.0643 | Val Loss: 0.0108 | Train Acc: 0.6329 | Val

Acc: 0.6298

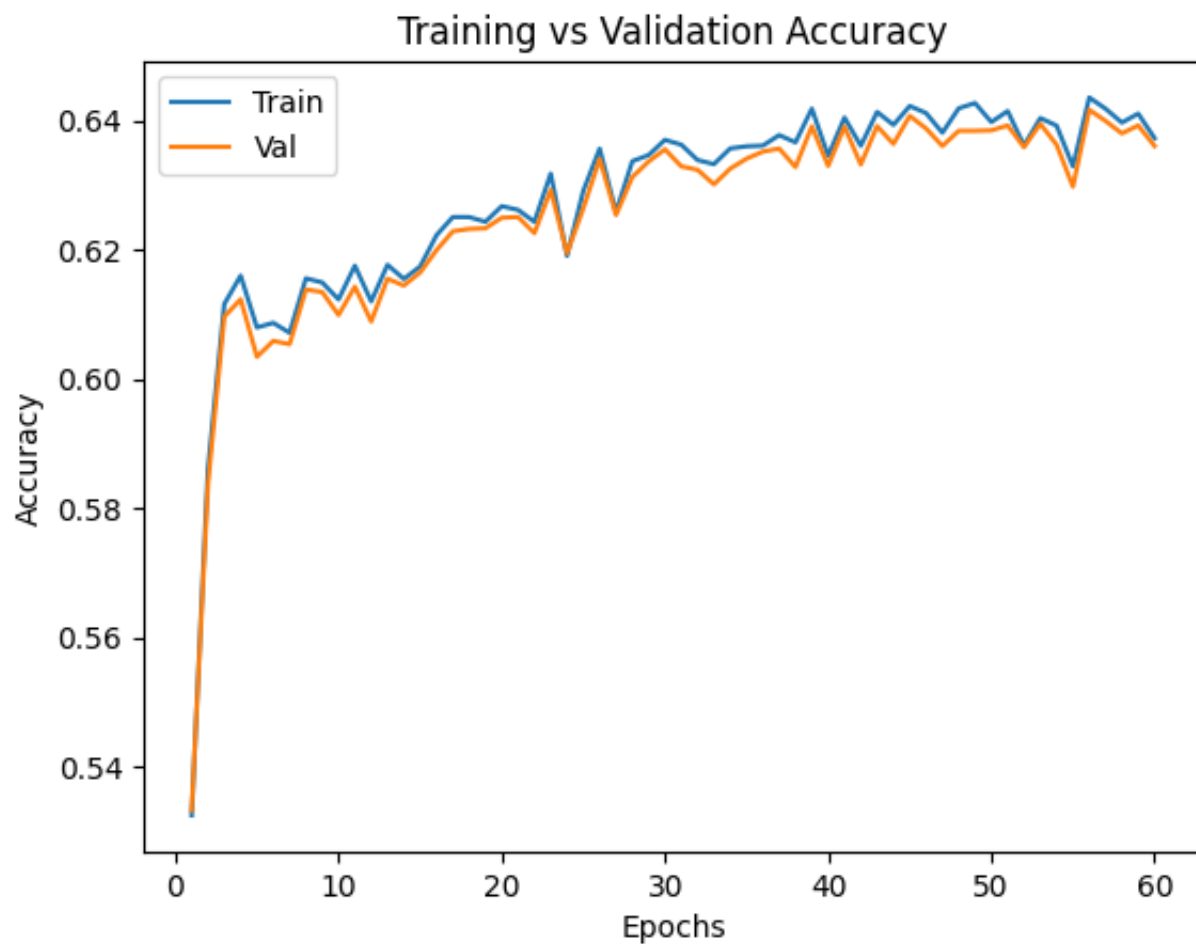
Epoch: 56 | Train Loss: 0.0684 | Val Loss: 0.0107 | Train Acc: 0.6436 | Val Acc: 0.6417

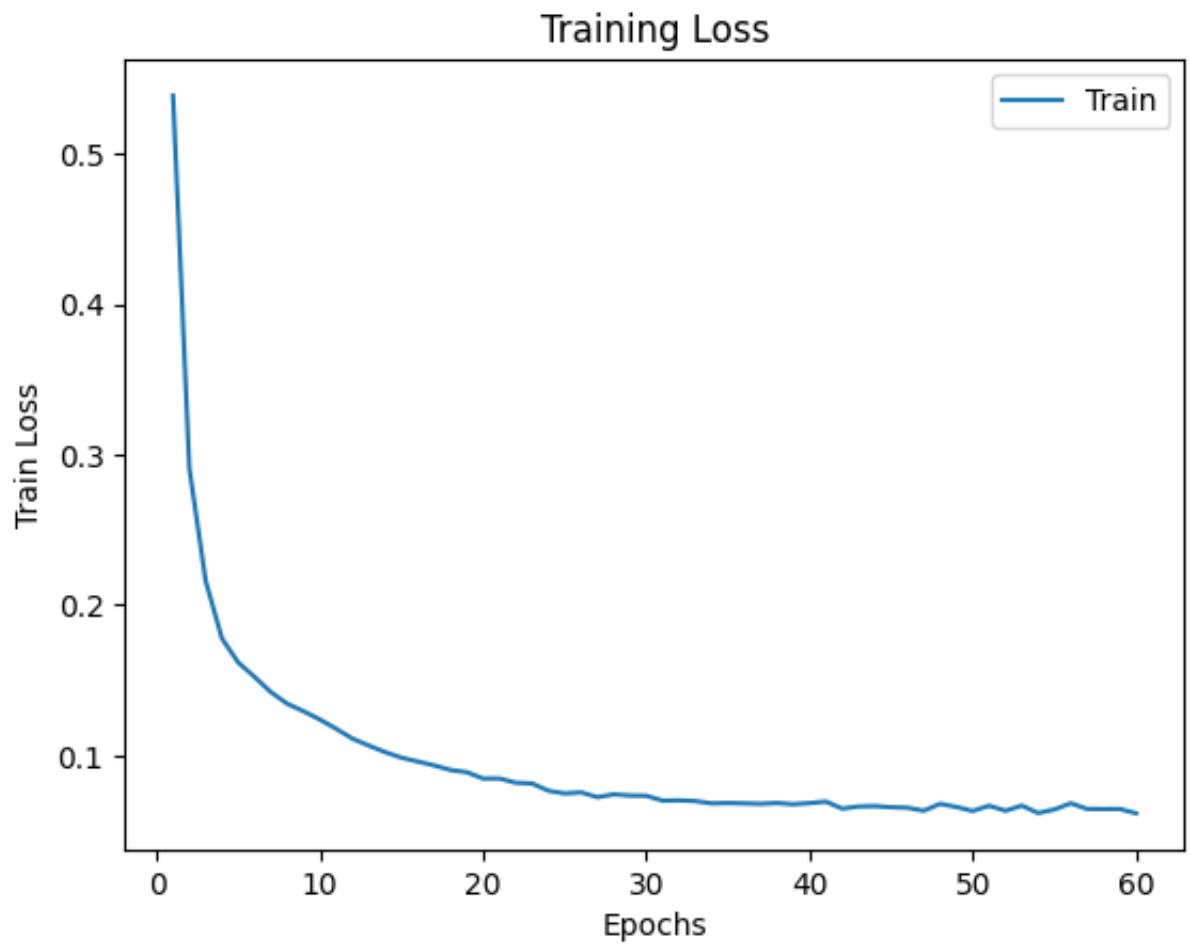
Epoch: 57 | Train Loss: 0.0646 | Val Loss: 0.0115 | Train Acc: 0.6418 | Val Acc: 0.6399

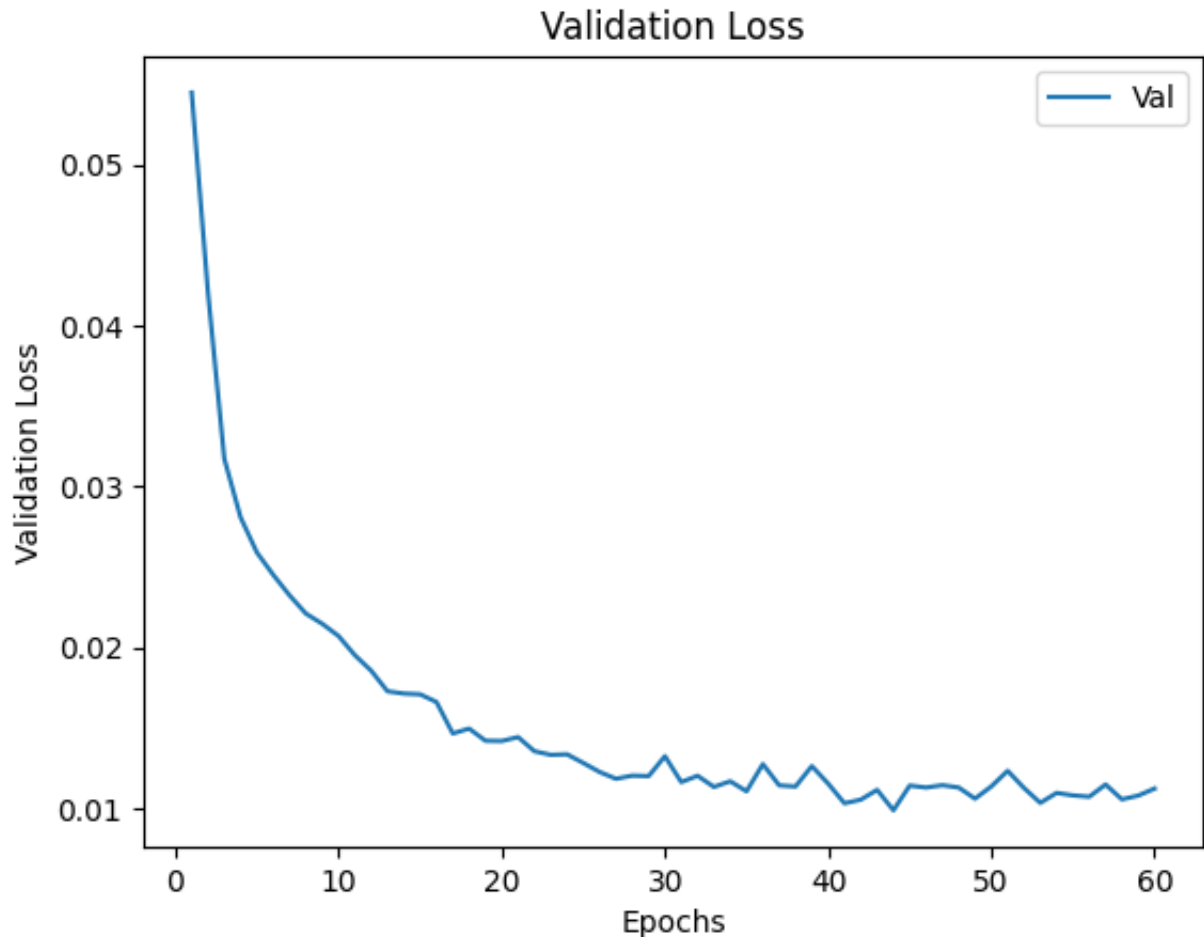
Epoch: 58 | Train Loss: 0.0645 | Val Loss: 0.0106 | Train Acc: 0.6397 | Val Acc: 0.6380

Epoch: 59 | Train Loss: 0.0646 | Val Loss: 0.0108 | Train Acc: 0.6411 | Val Acc: 0.6393

Epoch: 60 | Train Loss: 0.0617 | Val Loss: 0.0112 | Train Acc: 0.6373 | Val Acc: 0.6361







Part 4. Testing [12 pt]

Part (a) [2 pt]

Compute and report the test accuracy.

```
In [172... test_loader = torch.utils.data.DataLoader(test_set, batch_size=64, shuffle=True)
test_acc = get_accuracy(autoencoder1, test_loader)
print("The test accuracy of the best autoencoder model is", test_acc*100, "%")
```

The test accuracy of the best autoencoder model is 63.66464120370371 %

Part (b) [4 pt]

Based on the test accuracy alone, it is difficult to assess whether our model is actually performing well. We don't know whether a high accuracy is due to the simplicity of the problem, or if a poor accuracy is a result of the inherent difficulty of the problem.

It is therefore very important to be able to compare our model to at least one alternative.

In particular, we consider a simple **baseline** model that is not very computationally expensive. Our neural network should at least outperform this baseline model. If our network is not much better than the baseline, then it is not doing well.

For our data imputation problem, consider the following baseline model: to predict a missing feature, the baseline model will look at the **most common value** of the feature in the training set.

For example, if the feature "marriage" is missing, then this model's prediction will be the most common value for "marriage" in the training set, which happens to be "Married-civ-spouse".

What would be the test accuracy of this baseline model?

```
In [170... most_common = get_features(train_set.sum(axis = 0))

correct = 0
for record in test_set:
    test_values = get_features(record)
    for feature in most_common:
        if most_common[feature] == test_values[feature]:
            correct += 1

accuracy = correct/(len(test_set) * len(catcols))
print("The accuracy of the baseline model is", accuracy * 100, "%")
```

The accuracy of the baseline model is 45.68504050925926 %

Part (c) [1 pt]

How does your test accuracy from part (a) compared to your baseline test accuracy in part (b)?

The test accuracy from part a) was 63.66%, which is must higher compared to the baseline test accuracy which is 45.69%.

Part (d) [1 pt]

Look at the first item in your test data. Do you think it is reasonable for a human to be able to guess this person's education level based on their other features? Explain.

```
In [173... for col in catcols:
    print(col + ": " + get_categorical_value(get_onehot(test_set[0], col), col)
```

work: Private
marriage: Divorced
occupation: Prof-specialty
edu: Bachelors
relationship: Not-in-family
sex: Male

I think you could make an educated guess but not with high accuracy. Because he works in a prof specialty job it usually means more education so I would think that he would have a college degree. However, that's the only clue that's given and even then I wouldn't be able to accurately guess what level of education he has.

Part (e) [2 pt]

What is your model's prediction of this person's education level, given their other features?

```
In [177... out = autoencoder1(zero_out_feature(torch.tensor(test_set[0]).view(1,57), "e
print(get_categorical_value(get_onehot(out[0], "edu"), "edu"))
```

Masters

Part (f) [2 pt]

What is the baseline model's prediction of this person's education level?

```
In [178... start = cat_index["edu"]
stop = cat_index["edu"] + len(cat_values["edu"])
mode_idx = np.argmax(train_set.sum(axis=0)[start:stop])
print("edu:", cat_values["edu"][mode_idx])
```

edu: HS-grad